ARDUINO / ESP32 TUTORIAL (V2)

ESP32 Dual I2C: SSD1306 OLED & MPU6050 Code Analysis

Detailed line-by-line technical explanation of managing dual independent hardware I2C buses on ESP32

Code Overview & Technical Architecture

This Arduino sketch demonstrates how to run **two separate hardware I2C buses** on an ESP32 microcontroller simultaneously:

- **Primary I2C Bus (Wire - Bus 0):** Drives an SSD1306 128x64 OLED display using GPIO 21 (SDA) and GPIO 20 (SCL) at address `0x3C`.
- **Secondary I2C Bus (MPUWire - Bus 1):** Scans for an MPU6050 Accelerometer/Gyroscope module using GPIO 2 (SDA) and GPIO 1 (SCL) at address `0x68`.

1. Library Header Includes

LINE	CODE	DETAILED EXPLANATION
1	`#include <Wire.h>`	Includes the standard Arduino **Wire library**, providing the `TwoWire` class and basic API required for I2C communication.
2	`#include <Adafruit_GFX.h>`	Includes the core **Adafruit Graphics Library**, responsible for drawing primitives (lines, shapes) and handling text scaling/formatting.
3	`#include <Adafruit_SSD1306.h>`	Includes the hardware driver library specifically for **SSD1306 OLED displays**, mapping graphic commands to screen RAM buffers.

2. Macro Definitions & Bus Declarations

LINE	CODE	DETAILED EXPLANATION

5–6	`// ================ OLED ================`
	`#define SCREEN_WIDTH 128`
	`#define SCREEN_HEIGHT 64`
		Section header comment and preprocessor macros defining OLED screen dimensions (**128 pixels wide by 64 pixels high**).

8–9	`#define OLED_SDA 21` `#define OLED_SCL 20`	Defines ESP32 **GPIO 21** as SDA (Serial Data) and **GPIO 20** as SCL (Serial Clock) for the primary I2C bus connected to the OLED display.

11	`Adafruit_SSD1306 display(`
	`  SCREEN_WIDTH,`
	`  SCREEN_HEIGHT,`
	`  &Wire,`
	`  -1`
	`);`
		Instantiates the `display` object assigned to the standard `Wire` bus (Bus 0). The `-1` indicates no dedicated hardware reset pin is used.

LINE	CODE	DETAILED EXPLANATION
13–16	`// ================= MPU6050 ================= #define MPU_SDA 2 #define MPU_SCL 1 #define MPU6050_ADDR 0x68`	Section header and definitions for the MPU6050 sensor: assigns **GPIO 2** (SDA), **GPIO 1** (SCL), and sets default I2C address `0x68` (AD0 grounded).
18–19	`// Second I2C bus for MPU6050 TwoWire MPUWire = TwoWire(1);`	**CRITICAL FEATURE:** Instantiates a separate `TwoWire` object named `MPUWire` bound specifically to **Hardware I2C Peripheral Bus 1** of the ESP32, isolating it from the OLED's bus.

3. Setup Function & Peripheral Initialization

LINE	CODE	DETAILED EXPLANATION
22–23	`void setup() {  Serial.begin(115200);`	Begins the microcontroller initialization block and configures UART Serial communication at **115200 baud** for debugging console prints.
25–26	`// ================= OLED I2C ================= Wire.begin(OLED_SDA, OLED_SCL);`	Initializes standard I2C Bus 0 (`Wire`) on **GPIO 21 (SDA)** and **GPIO 20 (SCL)** for the OLED screen.
28–32	`if (!display.begin(  SSD1306_SWITCHCAPVCC,  0x3C )) {  Serial.println("OLED not found!");  while (1); }`	Initializes OLED hardware with internal charge pump power supply (`SSD1306_SWITCHCAPVCC`) at address `0x3C`. If missing/failed, prints error and halts in an infinite loop (`while(1)`).
34–35	`// ================= MPU6050 I2C ================= MPUWire.begin(MPU_SDA, MPU_SCL);`	Initializes the secondary hardware I2C bus (`MPUWire`) on **GPIO 2 (SDA)** and **GPIO 1 (SCL)** for the MPU6050 sensor.
37–39	`MPUWire.beginTransmission(MPU6050_ADDR); byte error = MPUWire.endTransmission();`	Performs an I2C bus ping/scan test on `MPUWire` to address `0x68`. `endTransmission()` returns `0` if an ACK response is received from the MPU6050.
41–45	`if (error == 0) {  Serial.println("MPU6050 found!"); } else {  Serial.println("MPU6050 not found!"); }`	Checks the ping result code. Prints `"MPU6050 found!"` to Serial if ACK succeeded (error 0), otherwise logs `"MPU6050 not found!"`.

4. Text Rendering & Framebuffer Display

LINE	CODE	DETAILED EXPLANATION
47–48	`// ================ OLED DISPLAY ================` `display.clearDisplay();`	Clears any existing data or splash screen memory stored inside the display's micro-buffer RAM.

50–53	`display.setTextSize(2);`
	`display.setTextColor(SSD1306_WHITE);`
	`display.setCursor(0, 0);`
	`display.println("Hello!");`
		Sets large text scaling (Size 2 = 12x16 px), sets active pixel color to WHITE, sets cursor position to top-left (0,0), and writes `"Hello!"` to buffer.

55–57	`display.setTextSize(1);`
	`display.setCursor(0, 30);`
	`display.println("ESP32 OLED");`
		Resets text scale back to normal (Size 1 = 6x8 px), moves render cursor down to Y=30, and writes `"ESP32 OLED"` to buffer.

59–60	`display.setCursor(0, 45);` `display.println("MPU6050 Ready");`	Moves render cursor further down to Y=45 and appends string `"MPU6050 Ready"` into RAM buffer.
62	`display.display();`	**CRITICAL STEP:** Sends complete rendered graphics/text buffer over I2C Bus 0 (`Wire`) to physically paint the OLED screen.
63	`}`	Ends the `setup()` execution block.

5. Main Loop Function

LINE	CODE	DETAILED EXPLANATION
66–67	`void loop() {` `}`	Main infinite execution loop left empty as this sketch performs static initialization and hardware ping check.

Hardware Pinout & Bus Connections

- **OLED Display (I2C Bus 0 - `Wire`):** SDA → ESP32 GPIO 21 | SCL → ESP32 GPIO 20
- **MPU6050 Sensor (I2C Bus 1 - `MPUWire`):** SDA → ESP32 GPIO 2 | SCL → ESP32 GPIO 1
- **Power Connections:** VCC → 3.3V / 5V | GND → Common GND